

Expense Management

Complete Code Documentation

This document explains every part of the Expense Management application: project structure, backend API, database, authentication, frontend UI, JavaScript logic, CSS styling, and how all components work together.

Tech Stack: Node.js, Express, SQLite, JWT, HTML/CSS/JavaScript

1. Project Overview

Expense Management is a full-stack web application that lets users sign up, log in, add/edit/delete expenses, view a sorted expense table, and see a pie chart of spending by category. Each user only sees their own data.

1.1 Architecture

The app uses a client-server architecture. The browser loads HTML/CSS/JS from the frontend folder. JavaScript sends JSON requests to REST API endpoints (/api/*). Express (Node.js) handles those requests, validates JWT tokens, and reads/writes data in a SQLite database file (backend/expenses.db).

1.2 Project Structure

Expense Management/

2. Dependencies (package.json)

The project uses npm to manage these packages:

Package	Purpose
express	Web server framework for API routes
better-sqlite3	Fast SQLite database driver
bcryptjs	Hash and verify passwords securely
jsonwebtoken	Create and verify JWT login tokens
cors	Allow cross-origin requests

Scripts: npm start runs the server. npm run dev runs with auto-restart on file changes.

3. Database Layer (backend/db.js)

This file creates and configures the SQLite database. When the server starts, better-sqlite3 opens backend/expenses.db and runs CREATE TABLE IF NOT EXISTS statements so tables exist automatically.

3.1 Users Table

- id - Auto-increment primary key
- email - Unique, required (stored lowercase)
- password - Bcrypt hash, never plain text
- created_at - Timestamp when account was created

3.2 Expenses Table

- id - Auto-increment primary key
- user_id - Links expense to a user (foreign key)
- category - e.g. Food, Transport, Bills
- amount - Positive number (REAL type)
- comments - Optional note text
- created_at / updated_at - Timestamps

The db object is exported and imported by index.js. All SQL uses prepared statements with ? placeholders to prevent SQL injection.

4. Authentication (backend/auth.js)

4.1 JWT Secret

JWT_SECRET comes from environment variable or defaults to 'expense-tracker-secret-key'. In production, always set a strong JWT_SECRET environment variable.

4.2 signToken(userId, email)

Creates a JSON Web Token containing userId and email. Token expires in 7 days. Returned to the client after signup or login.

4.3 authMiddleware

Protects expense routes. Reads Authorization: Bearer <token> header, verifies the JWT, and attaches req.user = { userId, email }. Returns 401 if missing, invalid, or expired token.

5. Backend API (backend/index.js)

Express app listens on PORT (default 3000). Uses cors() and express.json(). Serves frontend static files. All API routes start with /api.

5.1 Auth Routes

Method	Route / Description
POST	/api/auth/signup - Create account
POST	/api/auth/login - Log in and get token
GET	/api/health - Server health check

Signup flow:

- Validate email and password (min 6 chars)
- Normalize email to lowercase
- Hash password with bcrypt (10 rounds)
- Insert into users table; return JWT + email
- 400 if email already exists

Login flow:

- Find user by email
- Compare password with bcrypt.compareSync
- Return JWT + email or 401 Invalid email or password

5.2 Expense Routes (all require authMiddleware)

Method	Route / Description
GET	/api/expenses - List user expenses (newest first)
POST	/api/expenses - Add new expense
PUT	/api/expenses/:id - Update expense
DELETE	/api/expenses/:id - Delete expense
GET	/api/expenses/stats/by-category - Pie chart data

Every expense query filters by user_id from the JWT so users cannot access each other's data. Update sets updated_at to current time. Delete returns 404 if expense not found or belongs to another user.

6. Frontend HTML (frontend/index.html)

Single HTML file with four logical pages, toggled visible/hidden by JavaScript. No page reloads when switching views.

6.1 Pages

- page-login - Email/password login form
- page-signup - New account registration
- page-home - Dashboard: add expense form, total, expense table
- page-chart - Pie chart and category breakdown list

6.2 Key Elements

- form-login / form-signup - Authentication forms
- form-expense - Add or edit expense (category dropdown, amount, comments)
- expense-table - tbody filled dynamically by JavaScript
- pie-chart - CSS conic-gradient pie chart
- error-* elements - Show validation and API error messages

Categories: Food, Transport, Shopping, Bills, Entertainment, Health, Other.

7. Frontend CSS (frontend/css/style.css)

Modern, clean UI with card-based layout and responsive design.

7.1 Layout and Theme

- .hidden - display:none for page switching
- .card - White rounded boxes with shadow
- .auth-card - Centered login/signup card (max 400px)
- body - Light gray background (#f0f4f8)
- Primary color #0f3460, danger/error #e94560

7.2 Components

- .btn-primary / .btn-secondary / .btn-danger - Button styles
- .form-row - 3-column grid (1 column on mobile)
- table - Expense list with hover highlight
- .total-amount - Large bold total display
- .pie - 220px circle for conic-gradient chart
- .dot - Color legend dots next to chart labels

8. Frontend JavaScript (frontend/js/app.js)

8.1 Helper Functions

- showPage(name) - Hide all .page, show page-{name}
- showError(id, msg) - Display error message in element
- getToken / setAuth / clearAuth - localStorage JWT management
- formatDate / formatMoney - Display formatting
- api(path, method, body) - fetch wrapper for /api calls with Bearer token

8.2 api() Function

Central function for all backend communication. Sends JSON, attaches Authorization header if token exists, parses JSON response, throws Error with server message on failure.

8.3 loadExpenses()

Fetches GET /api/expenses, calculates total, builds table rows with Edit/Delete buttons, wires click handlers. Edit populates the form; Delete confirms then calls DELETE.

8.4 loadChart()

Fetches GET /api/expenses/stats/by-category. Builds CSS conic-gradient pie chart and legend list with percentages. Uses COLORS array for segment colors.

8.5 Event Handlers

- Login/Signup forms - POST to /api/auth/login or signup, save token, go to home
- Expense form - POST new or PUT edit based on editId variable
- Logout - Clear localStorage, show login page
- Chart/Back buttons - Switch between home and chart pages

8.6 Startup

On load: checkServer() pings /api/health. If token in localStorage, restore session and load expenses. Otherwise show login page.

9. Complete Data Flow

9.1 User Registration

```
Browser -> POST /api/auth/signup {email, password}
```

Server

9.2 Adding an Expense

```
Browser -> POST /api/expenses + Bearer token
```

Server

9.3 Viewing Chart

```
Browser -> GET /api/expenses/stats/by-category + Bearer token
```

Server

10. Security Features

- Passwords hashed with bcrypt - never stored in plain text
- JWT tokens expire after 7 days
- All expense routes require valid token
- SQL uses parameterized queries (prepared statements)
- Each user can only access their own expenses (user_id filter)

11. How to Run

```
npm install
```

npm

Important: Always use the server URL. Opening index.html directly in the browser will fail because /api requests have no server to connect to.

12. API Request/Response Examples

Signup Request:

```
POST /api/auth/signup
```

{ "email": "user@example.com", "password": "password" }

Signup Response (201):

```
{"token": "eyJhbGciOiJIbGciLCJ0eSI6InR5cGU6YXV0aCJ9", "email": "user@example.com"}
```

Create Expense Request:

```
POST /api/expenses
```


